

# Projektdokumentation – Book Buddy

---

Modul: M294 (Frontend) / M295 (Backend)

Name: Patrik Hafner

Abgabedatum: [19.07.2025]



## Inhaltsverzeichnis

|                                        |    |
|----------------------------------------|----|
| 1. Projektidee Elevator-pitch .....    | 3  |
| 2. Anforderungsanalyse .....           | 3  |
| 3. User Stories .....                  | 5  |
| 3.1 Buch erfassen .....                | 5  |
| 3.2 Bücher anzeigen .....              | 5  |
| 3.3 Buch bearbeiten .....              | 6  |
| 3.4 Buch löschen .....                 | 6  |
| 3.5 Kategorien verwalten .....         | 6  |
| 3.6 Freie Nutzung der Kategorien ..... | 7  |
| 4. Storyboard (Frontend) .....         | 7  |
| 5. Mockups .....                       | 9  |
| 6. Klassendiagramm .....               | 11 |
| 7. REST-Schnittstellen .....           | 11 |
| 7.1 Planung .....                      | 12 |
| 7.2 Dokumentation .....                | 12 |
| 8. Testing .....                       | 16 |
| 9. Installationsanleitung .....        | 18 |
| 10. Hilfestellungen .....              | 19 |

## 1. Projektidee Elevator-pitch

BookBuddy ist eine schlanke, benutzerfreundliche Webanwendung zur Verwaltung der eigenen Büchersammlung. Ohne unnötigen Ballast konzentriert sich die App auf das Wesentliche: Titel, Autor und Kategorie – mehr braucht es nicht, um Ordnung in die eigene Bibliothek zu bringen.

Ob gelesene Werke, geplante Lektüren oder Lieblingsklassiker – BookBuddy passt sich flexibel an deine persönliche Struktur an. Die Kategorien kannst du so nutzen, wie es für dich am sinnvollsten ist: als Status, Genre oder ganz frei nach deinem System.

Die Anwendung ist ideal für Leserinnen und Leser, die ihre Sammlung digital und unkompliziert verwalten möchten. Kein Account-Zwang, kein Social-Feature, kein Overhead – einfach nur Bücher eintragen, organisieren und im Blick behalten. Schnell. Klar. Effizient.

BookBuddy ist nicht mehr und nicht weniger als dein digitales Bücherregal – immer zur Hand, wenn du es brauchst.

## 2. Anforderungsanalyse

### 1. Problemstellung und Bedarf

Im digitalen Zeitalter existieren zahllose Apps rund um Bücher – viele davon jedoch mit überladenen Funktionen: Lesestatus-Tracking, Cover-Ansichten, Social Features, Rezensionen, Cloud-Anbindung. Für viele Nutzer, die lediglich eine einfache Möglichkeit zur strukturierten Erfassung ihrer persönlichen Büchersammlung suchen, sind solche Systeme überdimensioniert, unübersichtlich oder schlichtweg zu aufwendig.

**Was fehlt, ist eine minimalistische Lösung**, die sich auf das Wesentliche konzentriert:

- Welche Bücher besitze ich oder habe ich gelesen?
- Wer ist der Autor?
- Wie möchte ich diese Bücher kategorisieren (z. B. gelesen, ungelesen, Genre-basiert oder individuell)?

Dieser Bedarf besteht vor allem bei:

- Privatpersonen mit wachsender Buchsammlung
- Leserinnen und Leser, die analog lesen, aber digital verwalten möchten

- Personen, die bestehende Tools zu komplex oder zu restriktiv finden
  - Bildungseinrichtungen oder kleinen Organisationen ohne Bibliothekssystem
- 

## 2. Zielsetzung des Projekts

BookBuddy soll als **einfaches, schnelles und lokales Verwaltungstool** für Bücher dienen. Im Fokus stehen Übersichtlichkeit, Bedienkomfort und ein klar begrenzter Funktionsumfang. Es wird **bewusst auf Funktionen wie Lesestatus, Coverbilder, Bewertungen oder Benutzerkonten verzichtet**.

**Die Hauptfunktionen lauten:**

- Erfassen eines Buchs mit Titel, Autor und Kategorie
- Bearbeiten und Löschen vorhandener Einträge
- Übersicht aller Bücher in strukturierter Listenform

Diese Reduktion schafft eine intuitive Benutzererfahrung – insbesondere für Nutzer, die „einfach nur eine Liste ihrer Bücher“ pflegen möchten, ohne sich mit zusätzlichen Metadaten oder Registrierungspflichten zu beschäftigen.

---

## 3. Technischer Rahmen

Das System wird als clientseitige Webapplikation umgesetzt – initial ohne Persistenz, optional erweiterbar um Speicherung via localStorage, IndexedDB oder Anbindung an ein Backend. Es kommen moderne Webtechnologien wie **Vite** und **React** zum Einsatz, was eine performante und modulare Architektur gewährleistet.

---

## 4. Nutzen und Mehrwert

- **Einfachheit statt Funktionsüberfrachtung:** Klare Fokussierung auf die Kernbedürfnisse von Leserinnen und Lesern.
- **Niedrige Einstiegshürde:** Keine Anmeldung, keine Konfiguration, sofort einsatzbereit.
- **Flexibilität durch freie Kategorien:** Nutzer entscheiden selbst, wie sie ihre Sammlung strukturieren wollen.
- **Effizienz in der Bedienung:** Inline-Bearbeitung direkt in der Listenansicht vermeidet unnötige Seitenwechsel.

---

## 5. Fazit

BookBuddy adressiert gezielt die Lücke zwischen überkomplexen Buchverwaltungsplattformen und der völligen Abwesenheit strukturierter digitaler Organisation. Die Anwendung erfüllt einen realen Bedarf nach Einfachheit, Klarheit und Effizienz – insbesondere für all jene, die ihre Bücher offline lesen, aber online organisieren möchten.

## 3. User Stories

### 3.1 Buch erfassen

Als Benutzer:in  
möchte ich ein neues Buch mit Titel, Autor und Kategorie erfassen können,  
damit ich meine persönliche Büchersammlung digital dokumentieren kann.

#### Akzeptanzkriterien:

- Eingabemaske mit Feldern für Titel, Autor und Kategorie
- Titel, Autor und Kategorie sind Pflichtfelder
- Nach dem Speichern wird das Buch in der Übersicht angezeigt

---

### 3.2 Bücher anzeigen

Als Benutzer:in  
möchte ich eine übersichtliche Liste aller erfassten Bücher sehen,  
damit ich jederzeit nachvollziehen kann, welche Bücher ich bereits gespeichert habe.

#### Akzeptanzkriterien:

- Anzeige von Titel, Autor und Kategorie für jedes Buch
  - Die nach Erfassungszeitpunkt von oben nach unten sortiert
-

### 3.3 Buch bearbeiten

Als Benutzer:in

möchte ich die Angaben eines bereits erfassten Buches ändern können, damit ich fehlerhafte Informationen korrigieren oder Titel, Autor und Kategorie anpassen kann.

#### **Akzeptanzkriterien:**

- Bearbeitungsfunktion über die Buchliste zugänglich
  - Änderungen werden gespeichert und direkt in der Liste sichtbar
- 

### 3.4 Buch löschen

Als Benutzer:in

möchte ich ein Buch aus meiner Sammlung löschen können, damit meine Liste aktuell und aufgeräumt bleibt.

#### **Akzeptanzkriterien:**

- Jede Buchkarte oder Listeneintrag enthält eine Löschoption
  - Wenn man auf Löschen klickt, wird der Eintrag ohne nachfrage sofort gelöscht
- 

### 3.5 Kategorien verwalten

Als Benutzer:in

möchte ich eigene Kategorien anlegen und verwalten können, damit ich meine Bücher thematisch oder nach persönlichem System strukturieren kann.

#### **Akzeptanzkriterien:**

- Kategorien können aus dem Dropdownmenü ausgewählt werden
  - Beim Erfassen eines Buches und dem bearbeiten, kann eine existierende Kategorie ausgewählt werden
-

### 3.6 Freie Nutzung der Kategorien

Als Benutzer:in

möchte ich die Kategorien flexibel verwenden können damit ich meine Bücher nach meinem eigenen System organisieren kann.

#### Akzeptanzkriterien:

- Die App gibt feste Kategorien vor weitere Kategorien können in der Datenbank erstellt werden und sind nach Erstellung im Dropdownmenü sichtbar.
- Nutzer:innen können selbst entscheiden, wie sie die Kategorien einsetzen

## 4. Storyboard (Frontend)

### 1. Startseite – Bücherübersicht

#### Szene 1:

Der Benutzer betritt die Startseite der Anwendung „BookBuddy“.

Zentral platziert sind der Haupttitel „BookBuddy“ und die Unterüberschrift „Bücherübersicht“.

#### Darstellung:

Darunter erscheinen mehrere Buchkarten mit folgenden Informationen:

- Titel (fett hervorgehoben)
  - Autor
  - Kategorie
- Jeweils darunter zwei Schaltflächen: **Löschen** und **Bearbeiten**

#### Zweck:

Der Benutzer erhält eine strukturierte Übersicht aller eingetragenen Bücher.

Durch die Schaltflächen kann er direkt mit den einzelnen Einträgen interagieren, ohne zusätzlichen Navigationsaufwand.

---

### 2. Bearbeitungsmodus – Buch aktualisieren

### **Szene 2:**

Nach einem Klick auf „Bearbeiten“ erweitert sich der jeweilige Bucheintrag um ein Inline-Bearbeitungsformular.

#### **Inhalte des Formulars:**

- Eingabefeld für den Titel
- Eingabefeld für den Autor
- Auswahlfeld für die Kategorie
- Schaltflächen: **Speichern, Abbrechen und Löschen**

### **Zweck:**

Die Bearbeitung erfolgt direkt im Kontext des jeweiligen Buches.

Der Nutzer bleibt auf derselben Seite, was den Workflow beschleunigt und unnötige Navigationswechsel vermeidet.

---

## **3. Neues Buch hinzufügen**

### **Szene 3:**

Durch Auswahl des Navigationspunkts „Buch hinzufügen“ gelangt der Benutzer zu einem separaten Formular.

#### **Formularstruktur:**

- Eingabefeld für den Titel
- Eingabefeld für den Autor
- Dropdown-Menü zur Auswahl einer Kategorie
- Schaltflächen: **Buch hinzufügen** und **Abbrechen**

### **Zweck:**

Ein klar strukturiertes und reduziertes Formular erlaubt dem Benutzer, neue Bücher effizient zur Sammlung hinzuzufügen. Die Benutzerführung ist geradlinig und intuitiv gestaltet.

---

## **Design und Benutzerführung**

- **Farbschema:** Dunkler oder Heller Hintergrund je nach Systemeinstellung mit kontrastreichen Texten zur besseren Lesbarkeit.
- **Typografie:** Deutlich, sachlich und responsiv.

- **Layout:** Klare visuelle Trennung zwischen einzelnen Elementen. Zentrierte Ausrichtung sorgt für Übersichtlichkeit.
  - **Schaltflächen:** Einheitlich gestaltet, gut sichtbar, mit direkter Handlungszuordnung.
- 

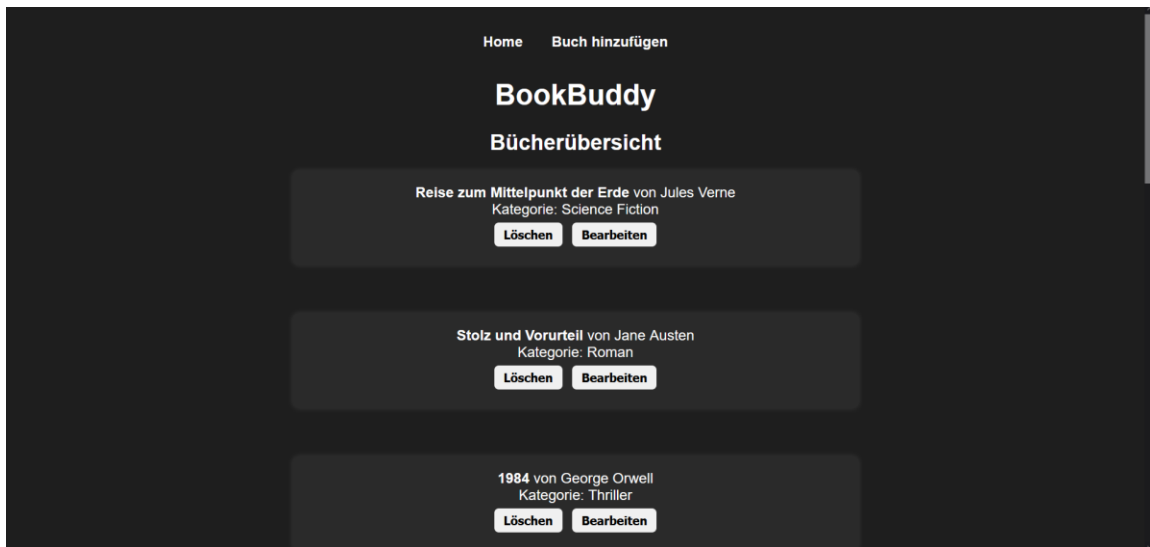
### Zielgruppe und Anwendungsszenarien

- Privatanwender zur Verwaltung der eigenen Bibliothek
  - Lehrpersonal oder Studierende zur Organisation von Leselisten
  - Kleine Organisationen oder Vereine mit Bedarf an einfacher Bucherfassung
- 

### Erweiterungspotenzial

- Implementierung einer Such- und Filterfunktion
- Unterstützung von Buchbewertungen und Notizen

## 5. Mockups



# BookBuddy

## Bücherübersicht

Reise zum Mittelpunkt der Erde von Jules Verne  
Kategorie: Science Fiction

[Löschen](#) [Bearbeiten](#)

Reise zum Mittelpunkt der Erde

Jules Verne

Science Fiction

[Speichern](#)

[Abbrechen](#)

Stolz und Vorurteil von Jane Austen

## Neues Buch hinzufügen

Titel:

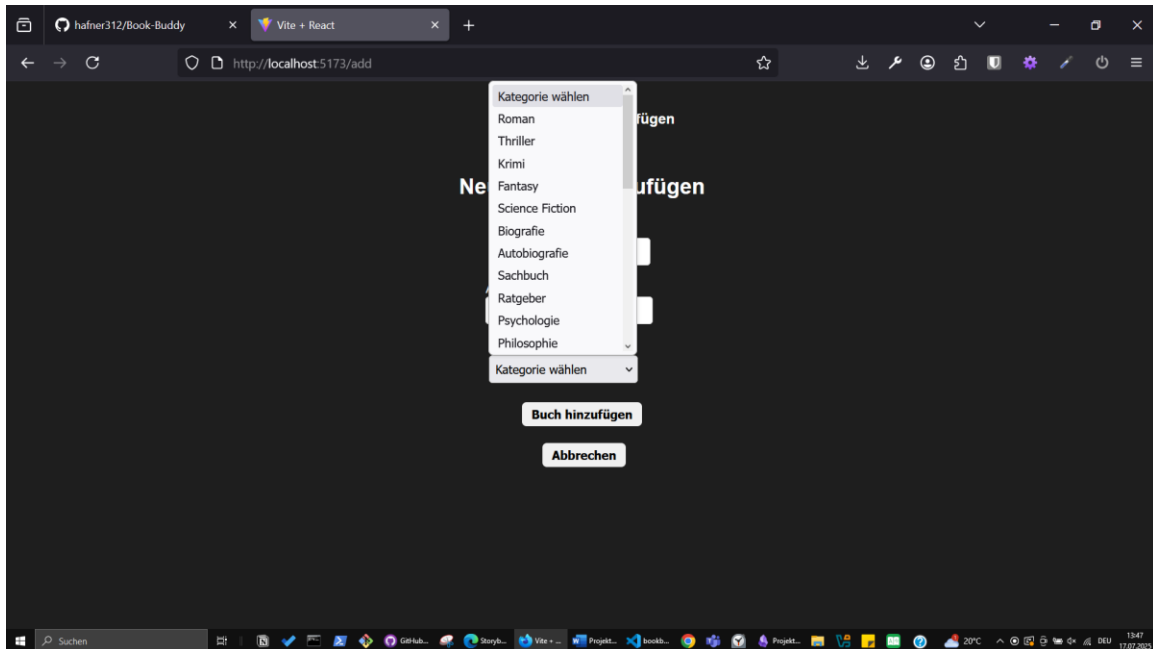
Autor:

Kategorie:

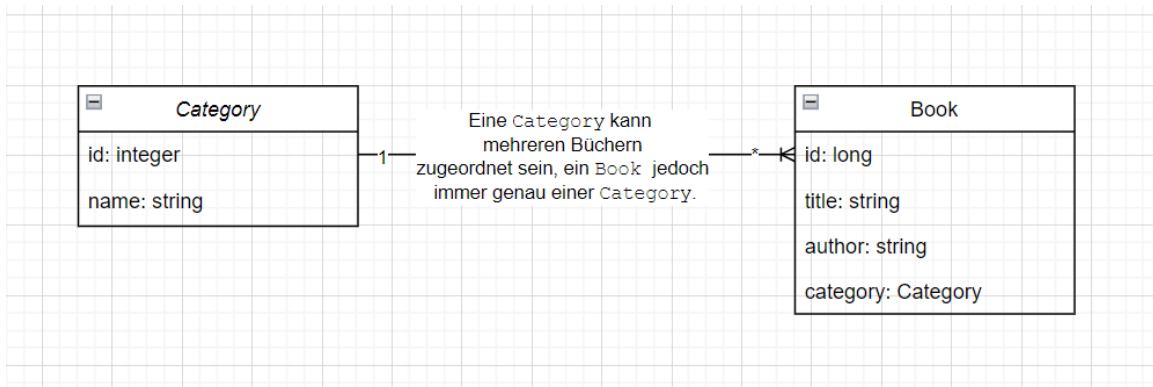
Kategorie wählen

[Buch hinzufügen](#)

[Abbrechen](#)



## 6. Klassendiagramm



In der Anwendung werden zwei Modellklassen verwendet, nämlich Book und Category. Ein Buch hat eine ID, einen Titel, einen Autor und gehört zu einer bestimmten Kategorie. Die Kategorie selbst besteht aus einer ID und einem Namen. Zwischen diesen beiden Klassen besteht eine Beziehung, bei der eine Kategorie mehreren Büchern zugeordnet sein kann, während ein Buch immer genau einer Kategorie angehört. Diese Struktur wurde im Klassendiagramm dargestellt. Dabei zeigt die Verbindungslinie vom Buch zur Kategorie an, dass es sich um eine sogenannte Many-to-One-Beziehung handelt. Die Attribute wurden mit ihren Datentypen ergänzt, um die Struktur der Klassen übersichtlich darzustellen. Dieses Modell bildet die Grundlage für die Speicherung und Verarbeitung der Daten im Backend.

## 7. REST-Schnittstellen

## 7.1 Planung

Um die Kommunikation zwischen dem Frontend und dem Backend sauber zu strukturieren, plane ich eine REST-API, die den Zugriff auf Bücher und Kategorien ermöglicht. Das Ziel ist es, dass die Benutzeroberfläche – in meinem Fall eine React-Anwendung – über klar definierte Schnittstellen mit dem Server kommunizieren kann. Dabei werden Standardmethoden wie GET, POST, PUT und DELETE verwendet, wie es beim REST-Prinzip üblich ist. Die Endpunkte werden unter `/api` gebündelt, um eine übersichtliche Struktur im Routing zu erhalten.

Für die Bücherverwaltung soll es möglich sein, alle Bücher als Liste abzurufen, ein neues Buch hinzuzufügen, ein bestehendes Buch zu aktualisieren und auch Bücher zu löschen. Ein Buch enthält die Felder Titel, Autor und Kategorie. Die Kategorie wird dabei über ihre ID zugewiesen. Alle Daten werden im JSON-Format zwischen Frontend und Backend ausgetauscht. Das macht die Integration in React sehr einfach und übersichtlich. Bei der Validierung der Eingaben (z. B. Pflichtfelder oder Längenbegrenzungen) setze ich auf serverseitige Prüfungen, um Datenkonsistenz sicherzustellen.

Zusätzlich zur Buchverwaltung wird auch eine einfache API für Kategorien benötigt. Diese wird verwendet, um im Frontend eine Auswahl an Kategorien anbieten zu können. Es soll einen GET-Endpunkt geben, über den alle vorhandenen Kategorien zurückgegeben werden. Zusätzlich plane ich eine Initialisierungsfunktion, die einmalig beim Projektstart eine vordefinierte Liste von Kategorien in die Datenbank schreibt. Dieser Schritt ist rein organisatorisch und wird im Live-Betrieb deaktiviert.

Insgesamt ist die API so aufgebaut, dass sie funktional klar abgegrenzt, leicht verständlich und problemlos erweiterbar ist. Falls im weiteren Verlauf noch Zeit ist oder sich ein Mehrwert ergibt, kann ich zusätzliche Funktionen ergänzen, etwa das Abrufen eines einzelnen Buchs, eine Suchfunktion oder ein Authentifizierungssystem zur Zugriffskontrolle. Vorrangig geht es aber darum, eine zuverlässige und nachvollziehbare Schnittstelle bereitzustellen, die das Frontend mit den Buch- und Kategoriedaten versorgt – schnell, effizient und wartbar.

## 7.2 Dokumentation

Im Rahmen der Anwendung *BookBuddy* wurden zwei REST-Controller implementiert, die den Zugriff auf Bücher und Kategorien ermöglichen. Die Schnittstellen basieren auf dem HTTP-Protokoll und folgen den Prinzipien eines RESTful API-Designs. Sämtliche Endpunkte sind unter `/api` gruppiert.

---

### 1. Bücher-API (`/api/books`)

Diese Schnittstelle stellt Funktionen zur Verfügung, um Bücher zu erfassen, anzuzeigen, zu bearbeiten und zu löschen.

### **GET /api/books**

- Gibt eine Liste aller gespeicherten Bücher zurück.
- Rückgabebetyp: List<Book>
- Beispielantwort:

json

KopierenBearbeiten

```
[  
  {  
    "id": 1,  
    "title": "1984",  
    "author": "George Orwell",  
    "category": {  
      "id": 3,  
      "name": "Science Fiction"  
    }  
  }  
]
```

### **POST /api/books**

- Erstellt ein neues Buchobjekt.
- Erwartet im Request-Body ein vollständiges Buchobjekt (inkl. Kategorie-ID).
- Rückgabebetyp: Book
- Beispielanforderung:

json

KopierenBearbeiten

```
{  
  "title": "Der Prozess",
```

```
"author": "Franz Kafka",  
"category": {  
  "id": 5  
}  
}
```

### **PUT /api/books/{id}**

- Aktualisiert ein bestehendes Buch anhand der ID.
- Erwartet im Request-Body ein vollständiges Buchobjekt.
- Rückgabetyp: Book
- Beispiel:

json

KopierenBearbeiten

```
{  
  "title": "Der Prozess (aktualisiert)",  
  "author": "Franz Kafka",  
  "category": {  
    "id": 5  
  }  
}
```

### **DELETE /api/books/{id}**

- Löscht ein Buch anhand der ID.
- Rückgabe: 204 No Content bei Erfolg, 404 Not Found wenn das Buch nicht existiert.

---

### **Datenstruktur: Book**

| <b>Feld</b> | <b>Typ</b> | <b>Validierung</b> |
|-------------|------------|--------------------|
|-------------|------------|--------------------|

| Feld     | Typ      | Validierung                      |
|----------|----------|----------------------------------|
| id       | Long     | Wird automatisch vergeben        |
| title    | String   | Pflichtfeld, max. 150 Zeichen    |
| author   | String   | Pflichtfeld, max. 100 Zeichen    |
| category | Category | Pflichtfeld, Referenz auf Entity |

---

## 2. Kategorien-API (/api/categories)

Diese Schnittstelle dient zur Verwaltung bzw. Initialisierung von Buchkategorien.

### GET /api/categories

- Gibt alle verfügbaren Kategorien zurück.
- Rückgabety: List<Category>
- Beispielantwort:

json

KopierenBearbeiten

```
[
  { "id": 1, "name": "Roman" },
  { "id": 2, "name": "Thriller" }
]
```

### POST /api/categories/init

- Legt eine Liste vordefinierter Kategorien an.
  - Wird typischerweise nur einmalig beim Systemstart verwendet.
  - Gibt eine Textmeldung mit dem Ergebnis zurück (neu angelegte oder bereits vorhandene Kategorien).
- 

## Datenstruktur: Category

## **Feld Typ Validierung**

id Integer Wird automatisch vergeben

name String Pflichtfeld, eindeutig, max. 100 Zeichen

---

## **Fehlerbehandlung und Statuscodes**

Die Anwendung verwendet folgende HTTP-Statuscodes zur Kommunikation mit dem Client:

### **Statuscode Bedeutung**

200 OK Anforderung erfolgreich verarbeitet

201 Created Objekt erfolgreich erstellt (optional)

204 No Content Objekt erfolgreich gelöscht

400 Bad Request Ungültiger Request-Body

404 Not Found Objekt mit angegebener ID existiert nicht

---

## **Zusätzliche Hinweise zur Verwendung**

- Die POST /api/categories/init-Schnittstelle dient ausschließlich der einmaligen Initialisierung und sollte im Live-Betrieb deaktiviert oder geschützt werden.
- Die Book- und Category-Objekte sind über JPA miteinander verknüpft (Many-to-One).
- Validierungen erfolgen über JSR-303-Annotationen (z. B. @NotBlank, @Size).

## **8. Testing**

### **Testkonzept und Umsetzung**

Zur Sicherstellung der grundlegenden Funktionalität und Zuverlässigkeit der Anwendung Book Buddy wurde ein pragmatischer Testansatz gewählt, der den Projektumfang und die gegebenen Ressourcen berücksichtigt. Dabei lag der Fokus auf der Prüfung der wichtigsten Kernfunktionen im Backend sowie auf manuellen End-to-End-Tests zur Absicherung der Gesamtfunktionalität.

## **Backend-Tests**

Im Backend wurden zwei zentrale automatisierte Tests implementiert. Diese prüfen die Geschäftslogik des BookService, insbesondere das Verhalten beim Speichern, Aktualisieren und Löschen von Büchern in Kombination mit gültigen oder ungültigen Kategorien. Dabei kamen JUnit 5 sowie Mockito zum Einsatz, um Repository-Abhängigkeiten zu simulieren. Zusätzlich wurde ein einfacher Start-Up-Test (Spring Boot Kontext-Test) durchgeführt, um sicherzustellen, dass die Anwendung korrekt initialisiert werden kann.

Weitere Unit- oder Integrationstests für Controller, Validierungslogik oder Fehlerbehandlung wurden im Rahmen dieses Projekts nicht umgesetzt. Die eingesetzte Bean Validation mit Annotationen wie `@NotBlank` und `@Size` in den Entitäten übernimmt grundlegende Prüfungen auf Eingabeebene. Ergänzt wird dies durch einen globalen Exception-Handler, der konsistente Rückmeldungen bei fehlerhaften Anfragen gewährleistet.

## **Frontend-Tests**

Im Frontend wurde bewusst auf automatisierte Tests verzichtet. Stattdessen wurden die wichtigsten Benutzerinteraktionen in den Formularen zur Bucherfassung und -bearbeitung manuell getestet. Dabei wurde besonderes Augenmerk auf die Validierung von Benutzereingaben, leere Felder und Fehlerzustände wie nicht erreichbare Server gelegt. Typische Fehlermeldungen wie "Bitte fülle alle Felder aus" oder "Verbindung zum Server fehlgeschlagen" wurden gezielt ausgelöst und auf ihre Benutzerfreundlichkeit geprüft.

Ein vorbereitetes Testmodul war im Projekt ursprünglich angelegt, wurde jedoch im Laufe der Entwicklung entfernt, da automatisierte Frontend-Tests für diesen Projektumfang nicht als priorisiert betrachtet wurden.

## **Ende-zu-Ende-Prüfung**

Die Kommunikation zwischen Frontend und Backend wurde durch vollständige Abläufe manuell getestet. Dazu gehörten Szenarien wie das Anlegen eines Buches, das Speichern über die REST-Schnittstelle, die Datenpersistenz in der Datenbank sowie die korrekte Anzeige in der Benutzeroberfläche. Durch diese vollständigen Funktionsdurchläufe konnte die Integration beider Module sichergestellt werden.

## **Fazit**

Die Teststrategie wurde so gewählt, dass sie bei begrenztem Umfang eine sinnvolle Balance zwischen Aufwand und Qualitätssicherung bietet. Die wichtigsten Funktionalitäten wurden erfolgreich getestet, potenzielle Fehlerquellen werden zuverlässig erkannt und Rückmeldungen benutzerfreundlich ausgegeben. Die Anwendung ist strukturell so aufgebaut, dass eine Erweiterung der Testabdeckung jederzeit möglich ist.

## 9. Installationsanleitung

Laden Sie die ZIP-Datei herunter und entpacken Sie sie vollständig. Öffnen Sie anschließend den entpackten Ordner mit Visual Studio Code. Das Projekt besteht aus zwei Teilen: dem Java-Backend (Ordner „backend“) und dem React-Frontend (Ordner „Front-End“).

Stellen Sie sicher, dass folgende Programme installiert sind:

- Java JDK Version 17 oder höher
- Node.js ab Version 18
- Maven
- Ein lokaler MySQL-Server (z. B. über XAMPP, MAMP oder eine eigenständige MySQL-Installation)

Bevor Sie das Projekt starten, muss die Datenbank vorbereitet werden:

1. Öffnen Sie **MySQL Workbench** oder ein vergleichbares Tool.
2. Führen Sie das mitgelieferte SQL-Skript **bookbuddy\_init.sql** aus, das sich im Projektordner befindet.  
Dieses Skript erstellt automatisch die Datenbank **bookbuddy**, legt die Tabellen **category** und **book** an und füllt sie mit Beispieldaten.  
Es sind bereits 27 Buchkategorien sowie mehrere Bücher enthalten – Sie müssen nichts manuell eingeben.
3. Öffnen Sie nun in Visual Studio Code die Datei **application.properties** im Ordner backend/src/main/resources.  
Tragen Sie dort Ihre lokalen MySQL-Zugangsdaten wie folgt ein:

- Datenbank-URL: jdbc:mysql://localhost:3306/bookbuddy
- Benutzername: z. B. root
- Passwort: Ihr individuelles MySQL-Passwort
- spring.jpa.hibernate.ddl-auto muss auf update gesetzt sein

4. Öffnen Sie ein Terminal in Visual Studio Code, wechseln Sie in den Ordner **backend** und starten Sie das Backend mit dem Befehl:  
**mvn spring-boot:run**
5. Öffnen Sie ein zweites Terminal, wechseln Sie in den Ordner **Front-End** und führen Sie die folgenden Befehle nacheinander aus:  
Zuerst: **npm install** (einmalig zur Installation aller Abhängigkeiten)  
Dann: **npm run dev** (zum Starten des Frontends)

Die Anwendung ist anschließend im Browser unter der Adresse <http://localhost:5173> erreichbar. Das Backend läuft im Hintergrund auf Port **8080** und ist mit der MySQL-Datenbank verbunden. Beim Start sehen Sie direkt die eingebundenen Kategorien und Bücher aus dem SQL-Skript.

---

**Hinweis:** Bitte führen Sie das SQL-Skript vor dem ersten Start unbedingt vollständig aus – andernfalls fehlen Daten oder Tabellen, und die Anwendung funktioniert nicht wie vorgesehen.

## 10. Hilfestellungen

- Hilfe durch den Lehrer der Module 294/295, Patrik Meier.
- Chat GPT
- Aufgeschnappte Konversationen während des Unterrichts und der Pausen.